

Resolución: El Problema de la Partición mediante Programación Dinámica

Este documento presenta la formalización, el diseño algorítmico y el análisis de complejidad para resolver el Problema de la Partición utilizando la metodología de Programación Dinámica.

1. Reducción y Análisis del Problema

Dado un conjunto (o multiset) de n números enteros positivos $A = \{a_1, a_2, \dots, a_n\}$, queremos determinar si existe una partición de A en dos subconjuntos disjuntos A_1 y A_2 tales que:

$$A_1 \cup A_2 = A, \quad A_1 \cap A_2 = \emptyset, \quad \sum_{x \in A_1} x = \sum_{y \in A_2} y$$

Propiedad Clave

Definamos la suma total de todos los elementos del conjunto como S :

$$S = \sum_{i=1}^n a_i$$

Si logramos dividir el conjunto en dos partes con la misma suma, cada subconjunto debe sumar exactamente la mitad de la suma total:

$$\text{Suma de } A_1 = \text{Suma de } A_2 = \frac{S}{2}$$

De esta observación deducimos dos condiciones críticas:

1. **Condición de Paridad:** Si la suma total S es un número impar, es matemáticamente imposible dividirla en dos partes enteras iguales. En este caso, el algoritmo puede devolver de inmediato que la partición no existe.
2. **Reducción al Problema de la Suma de Subconjuntos:** Si la suma total S es par, el problema se reduce exactamente a encontrar si existe un subconjunto de A cuya suma sea exactamente igual a la mitad de la suma total:

$$H = \frac{S}{2}$$

2. Ecuación en Recurrencias

Definimos la función booleana $P[i, w]$ que representa si es posible obtener una suma exacta de w ($0 \leq w \leq H$) utilizando únicamente un subconjunto de los primeros i elementos de la

entrada ($0 \leq i \leq n$).

Casos Base

1. **Suma objetivo igual a cero ($w = 0$):** Siempre es posible obtener una suma de 0 seleccionando el conjunto vacío, sin importar cuántos elementos tengamos disponibles:

$$P[i, 0] = \text{Verdadero} \quad \forall i \in \{0, 1, \dots, n\}$$

2. **Sin elementos disponibles ($i = 0$) para una suma positiva ($w > 0$):** Es imposible formar una suma mayor que cero si no tenemos ningún elemento:

$$P[0, w] = \text{Falso} \quad \forall w \in \{1, 2, \dots, H\}$$

Caso General (Paso de Recurrencia)

Para calcular $P[i, w]$ con $1 \leq i \leq n$ y $1 \leq w \leq H$, evaluamos la decisión de incluir o excluir el i -ésimo elemento a_i :

- **Si el elemento actual es mayor que la suma objetivo ($a_i > w$):** No podemos incluirlo en nuestro subconjunto. Por lo tanto, el resultado depende únicamente de las decisiones tomadas con los elementos anteriores:

$$P[i, w] = P[i - 1, w]$$

- **Si el elemento cabe ($a_i \leq w$):** Tenemos dos alternativas:
 - *No incluirlo:* Mantener el resultado sin el elemento actual ($P[i - 1, w]$).
 - *Incluirlo:* Buscar si es posible sumar la diferencia restante ($w - a_i$) con los elementos anteriores ($P[i - 1, w - a_i]$).

La relación de recurrencia es la disyunción lógica de ambas opciones:

$$P[i, w] = P[i - 1, w] \vee P[i - 1, w - a_i]$$

3. Algoritmo Iterativo con Tabla Bidimensional (Fácil de entender)

Para que el algoritmo sea lo más intuitivo posible, utilizaremos una matriz bidimensional DP de tamaño $(n + 1) \times (H + 1)$.

Cada fila i de la matriz representa los elementos que podemos usar, y cada columna w representa la suma que queremos conseguir. De esta forma, cada celda de la tabla se calcula directamente mirando las celdas de la fila de arriba ($i - 1$), tal y como describe la ecuación matemática.

Algoritmo en Pseudocódigo

```

Funcion ExisteParticionEquitativa(A, n)
Inicio
    // 1. Calcular la suma total de los elementos
    S <- 0
    Para i <- 1 Hasta n Con Paso 1 Hacer
        S <- S + A[i]
    FinPara

    // 2. Condición de paridad: Si la suma es impar, no se puede partir en dos
    Si S Mod 2 != 0 Entonces
        Retornar Falso
    FinSi

    H <- S / 2 // Objetivo de suma para cada uno de los dos conjuntos

    // 3. Crear la tabla bidimensional DP de tamaño (n + 1) x (H + 1)
    DP <- Matriz de dimensiones (n + 1) x (H + 1) inicializada en Falso

    // 4. Inicializar Casos Base en la tabla
    // Para cualquier cantidad de elementos 'i', siempre podemos conseguir la sum
    Para i <- 0 Hasta n Con Paso 1 Hacer
        DP[i, 0] <- Verdadero
    FinPara

    // 5. Rellenar la tabla celda por celda utilizando la recurrencia
    Para i <- 1 Hasta n Con Paso 1 Hacer
        Para w <- 1 Hasta H Con Paso 1 Hacer

            Si A[i] > w Entonces
                // El elemento actual es demasiado grande, no lo podemos usar.
                // Heredamos el resultado de no usarlo (celda justo arriba)
                DP[i, w] <- DP[i - 1, w]
            Sino
                // El elemento cabe. El resultado es verdadero si podíamos lograr
                // sin él (DP[i-1, w]) O si podíamos lograr la suma restante usan
                // de apoyo (DP[i-1, w - A[i]])
                DP[i, w] <- DP[i - 1, w] o DP[i - 1, w - A[i]]
            FinSi

        FinPara
    FinPara

    // El resultado final que nos dice si es posible conseguir la mitad H
    // usando los n elementos se encuentra en la esquina inferior derecha
    Retornar DP[n, H]
FinFuncion
.

```

6. Variante: Punto de Corte o Partición Física (Split Point) con Programación Dinámica

Diferencia conceptual con la versión anterior

En la partición general, buscamos subconjuntos dispersos formados por cualquier combinación de elementos. En esta variante de **Punto de Corte**, no podemos alterar el orden físico de la lista A y buscamos un único índice de corte k ($1 \leq k < n$) tal que:

$$\sum_{i=1}^k A[i] = \sum_{j=k+1}^n A[j]$$

Formulación formal por Programación Dinámica

Este problema se puede modelar de manera muy elegante con Programación Dinámica utilizando el concepto de **sumas acumuladas** mediante subproblemas solapados.

1. Definición de las Tablas DP

- $DP_{izq}[i]$: Almacena la suma acumulada de la secuencia desde la posición de inicio (1) hasta el índice i .
- $DP_{der}[i]$: Almacena la suma acumulada de la secuencia desde el final (n) de vuelta hacia atrás hasta el índice i .

2. Ecuaciones de Recurrencia de la Dinámica

Las sumas de los tramos se pueden definir de forma recursiva basándonos en los subproblemas inmediatamente anteriores o posteriores ya calculados (memorizados):

- Suma de prefijos (de izquierda a derecha):

$$DP_{izq}[i] = \begin{cases} 0 & \text{si } i = 0 \\ DP_{izq}[i - 1] + A[i] & \text{si } 1 \leq i \leq n \end{cases}$$

- Suma de sufijos (de derecha a izquierda):

$$DP_{der}[i] = \begin{cases} 0 & \text{si } i = n + 1 \\ DP_{der}[i + 1] + A[i] & \text{si } 1 \leq i \leq n \end{cases}$$

3. Condición de Optimalidad (Corte)

Una vez construidas ambas tablas DP, el punto de corte óptimo k es aquel índice que satisface la igualdad:

$$DP_{izq}[k] = DP_{der}[k + 1]$$

Algoritmo en Pseudocódigo

```

Funcion EncontrarPuntoDeCorteDinamico(A, n)
Inicio
    // 1. Crear las tablas de programación dinámica
    DP_izq <- Vector de tamaño n + 1
    DP_der <- Vector de tamaño n + 2

    // 2. Establecer casos base

```

```

DP_izq[0] <- 0
DP_der[n + 1] <- 0

// 3. Rellenar DP_izq (de izquierda a derecha)
Para i <- 1 Hasta n Con Paso 1 Hacer
    DP_izq[i] <- DP_izq[i - 1] + A[i]
FinPara

// 4. Rellenar DP_der (de derecha a izquierda)
Para i <- n Hasta 1 Con Paso -1 Hacer
    DP_der[i] <- DP_der[i + 1] + A[i]
FinPara

// 5. Buscar el punto de equilibrio k utilizando las tablas DP
Para k <- 1 Hasta n - 1 Con Paso 1 Hacer
    Si DP_izq[k] = DP_der[k + 1] Entonces
        Retornar k // Retorna el índice que delimita el final del primer subc
    FinSi
FinPara

// Si se recorre todo y no hay igualdad, no es posible hacer la partición
Retornar -1
FinFuncion

```

Análisis de Costes de la Variante Dinámica

- **Complejidad Temporal:** $O(n)$. Rellenar la tabla `DP_izq` toma un tiempo de $O(n)$ (un bucle simple), rellenar `DP_der` toma $O(n)$ (un bucle simple) y buscar el índice de coincidencia realiza a lo sumo $n - 1$ comprobaciones constantes. El coste temporal total es estrictamente lineal.
- **Complejidad Espacial:** $O(n)$. Almacenamos dos tablas adicionales de tamaño n (`DP_izq` y `DP_der`) para albergar los resultados intermedios de las sumas parciales, lo cual respeta la naturaleza clásica de almacenamiento de datos de la programación dinámica.